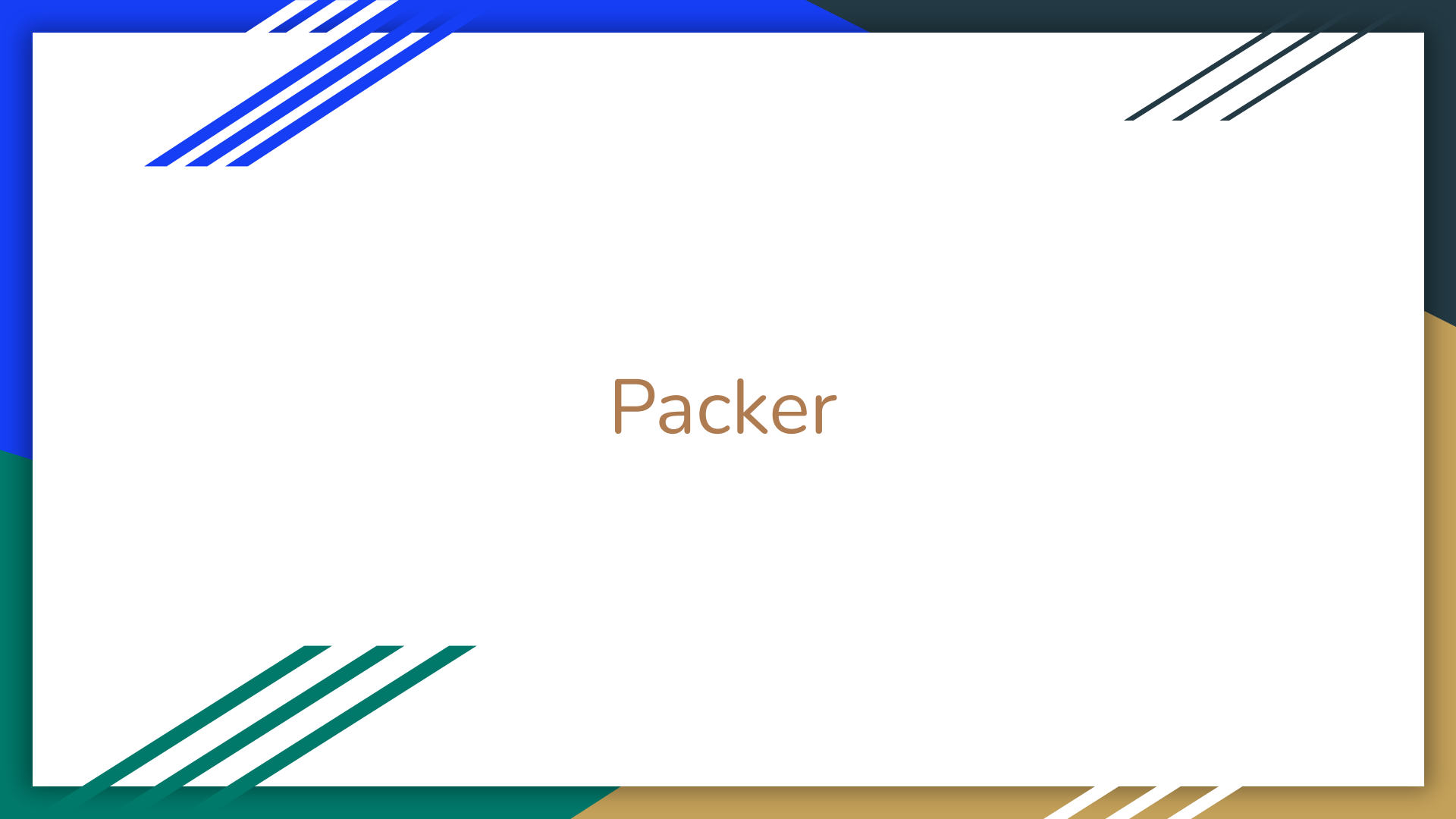




Packer

Installing Packer on Workstation

You can have the packer installation on your local workstation or on a cloud instance. All you need is the cli connectivity to AWS using access keys.

Note: If you are planning to setup Packer on the local workstation, make sure you have the aws access keys set on file `~/.aws/credentials`. It is not mandatory to have credentials file. You can pass the AWS credentials using the packer variable declaration.

1. Download the required package from here. <https://www.packer.io/downloads.html>
2. Unzip the package and set the path variable in `~/.bashrc`

```
export PATH=$PATH:/path/to/packer
```

Installing Packer on Workstation

3. Refresh terminal

```
source ~/.bashrc
```

4. Verify packer installation by executing the packer command.

```
packer version
```

Building Image

Packer configuration templates are written in JSON format.

A template has the following three main parts.

1. **variables** – Where you define custom variables.
2. **builders** – Where you mention all the required AMI parameters.
3. **provisioners** – Where you can integrate a shell script, ansible play or a chef cookbook for configuring a required application in the AMI.

An example template for packaging an AWS AMI is given below.

```
{
  "variables": {
    "aws_access_key": "",
    "aws_secret_key": ""
  },
  "builders": [{
    "type": "amazon-ebs",
    "access_key": "{{user `aws_access_key`}}",
    "secret_key": "{{user `aws_secret_key`}}",
    "region": "us-west-1",
    "source_ami": "ami-sd3543ds",
    "instance_type": "t2.medium",
    "ssh_username": "ec2-user",
    "ami_name": "packer-demo {{timestamp}}"
  ]},
  "provisioners": [
    {
      "type": "shell",
      "script": "demo-script.sh"
    }
  ]
}
```

Building Image

In the above example configuration, we are passing the AWS access keys and secret keys as variables. It is not recommended to use access keys as variables. If you have already set the credentials in `~/.aws/credentials` file, you don't need to pass access keys as variables.

Also, we have used shell provisioner which calls a `demo-script.sh` file.

packer supports the following provisioners.

- 1 Ansible
- 2 Chef
- 3 Salt
- 4 Shell
- 5 Powershell
- 6 Windows cmd
- 7 File – For copying file from local directory to VM image.

Building a Packer Template

To build a packer template, you just need to execute the `build` command with the JSON template.

For example, `packer build apache.json`

Before jumping into building the template, let's look at some more important concepts and at last we can try out an example with shell provisioner.

Different Scenarios for Using Variables

You can make use for packer variables for dynamic configuration fo packaging AMI. Let's discuss these scenarios one by one.

Using Variable Within Template

The `variables` block holds all the default variables within a template. Asn example is shown below.

```
"variables": {  
  "instance_type": "t2.medium",  
  "region": "us-west-1"  
}
```

Different Scenarios for Using Variables

The declared variables can be accessed in other parts of the template using `"{{user `your-variable-name`}}"` syntax. An example is shown below.

```
"instance_type": "{{user `instance_type`}}",  
"region": "{{user `region`}}"
```


Using Environment Variables in Templates

Packer lets you use the system environment variables. First, you need to declare the environment variables in the variable section to use it in the other parts of the template.

Lets say, you want to use the `SCRIPT_PATH` environment variable that holds the path to a shell script that has to be used in the shell provisioner. You can declare that variable like shown below.

```
"variables": {  
  "script_path": "{{env `SCRIPT_PATH`}}",  
}
```

Using Environment Variables in Templates

After the declaration, you can use the `script_path` variable in the provisioner like shown below.

```
"provisioners": [  
  {  
    "type": "shell",  
    "script": "{{user `script_path` }}/demo-script.sh"  
  }  
]
```

Using Command Line Variables

Using Command Line Variables

```
"app_name": "{{app_name_cmd_var}}"
```

You can pass variables during the run time using `-var` flag followed by the variable declaration.

You can use this variable in the template using the normal interpolation we used in the above examples.

For example,

```
packer build -var 'app_name_cmd_var=apache' apache.json
```

Using a JSON File

You can pass a JSON file with variables block to the build command as shown below.

```
packer build -var-file=variables.json apache.json
```

In the above example, `variables.json` is the variable file and `apache.json` is the packer template.

Packaging an Image

In this example, we will bake a t2.micro AMI using a shell provisioner. A shell script which has an update and HTTPD install instruction.

We assume that you have the AWS access keys and region set in the `~/.aws/credentials` file.

Here we are going to use `Oregon` region and a Redhat AMI with AMI id `ami-6f68cf0f`

Follow the steps given below to set up the project.

1. Create a folder call packer

```
mkdir packer
```

Packaging an Image

2. Create a script file named `demo-script.sh` and copy the following contents on to it.

```
#!/bin/bash  
sudo yum -y update  
sudo yum install -y httpd
```

The above script just updates the repository and installs httpd.

Packaging an Image

3. Create a `httpd.json` file with the following contents. 

We have our template ready. Next step is to execute the template to for packaging the AMI with HTTPD installation.

4. Lets validate and inspect our template using the following commands.

```
packer validate httpd.json
packer inspect httpd.json
```

Note: If you are using command line variables or a file as a variable, you should pass it while validating it. An example is shown below.

```
packer validate -var='env=TEST' httpd.json
```

```
{
  "variables": {
    "ami_id": "ami-6f68cf0f",
    "app_name": "httpd"
  },

  "builders": [{
    "type": "amazon-efs",
    "region": "eu-west-1",
    "source_ami": "{{user `ami_id`}}",
    "instance_type": "t2.micro",
    "ssh_username": "ec2-user",
    "ami_name": "PACKER-DEMO-{{user `app_name` }}",
    "tags": {
      "Name": "PACKER-DEMO-{{user `app_name` }}",
      "Env": "DEMO"
    }
  }],

  "provisioners": [
    {
      "type": "shell",
      "script": "demo-script.sh"
    }
  ]
}
```

Packaging an Image

5. To build our new AMI, use the following command.

```
packer build httpd.json
```

The above command will build a new AMI. You can also debug the image creation.

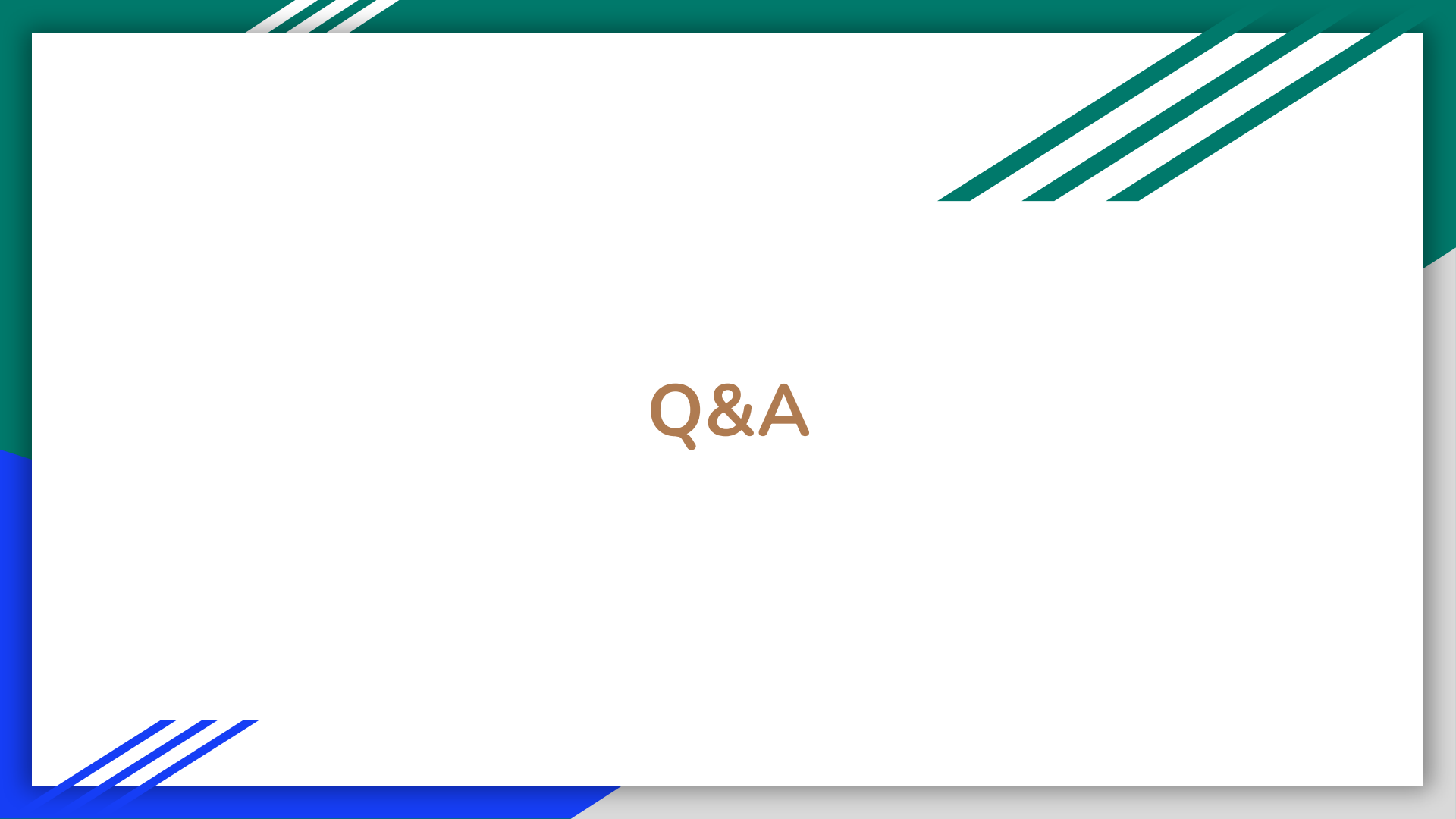
Few Tips

1. You can capture the output of the image build to a file using the following command.

```
packer build httpd.json 2>&1 | sudo tee output.txt
```

2. Then you can extract the new AMI id to a file named ami.txt using the following command.

```
tail -2 output.txt | head -2 | awk 'match($0, /ami-.*/) { print  
substr($0, RSTART, RLENGTH) }' > sudo ami.txt
```



Q&A